

String to Integer (atoi) — LeetCode 8

Problem

Implement `myAtoi(String s)`, which converts a string to a 32-bit signed integer.

1. **Read in and ignore leading whitespace.**
2. **Check if the next character is `-` or `+`.** If neither, assume the number is positive. (At most one sign character.)
3. **Read in the next characters until a non-digit is hit or the end of the string is reached.** The rest of the string is ignored.
4. **Convert** the digits into an integer. Change the sign as necessary (from step 2).
5. **Clamp:** if the integer is out of the 32-bit signed integer range `[-231, 231 - 1]`, clamp to `Integer.MIN_VALUE` / `Integer.MAX_VALUE`.
6. If there are no digits, return `0`.

Key observations (read the spec carefully)

- Only the ASCII space `' '` is skipped — **not** tabs, newlines, or other whitespace. Many solutions use `Character.isWhitespace`; the spec says only `' '`.
- The sign must come **immediately** after whitespace. `" + 413"` $\rightarrow$ `0`, because `'+'` is followed by a space, not a digit.
- There can be at most one sign. `"+-12"` $\rightarrow$ `0`, `"-+1"` $\rightarrow$ `0`.
- Leading zeros are fine: `"0032"` $\rightarrow$ `32`. A sign alone (`"+"`, `"-"`) $\rightarrow$ `0`.
- Clamping happens **during** accumulation, not after — the intermediate value never overflows `Long`.

Approach

A single left-to-right pass with an index pointer:

1. Advance `i` past leading spaces. If we hit the end, return `0`.
2. Read one optional sign character.
3. Accumulate digits into a `long`. After adding each digit, check the bounds:

- positive overflow $\rightarrow$ return `Integer.MAX_VALUE`

- negative overflow (`-result < Integer.MIN_VALUE`) $\rightarrow$ return `Integer.MIN_VALUE`

1. Return `(int)(result * sign)`.

The per-digit clamp means we never need arbitrary-precision arithmetic — the `long` accumulator only ever needs to hold one digit more than the 32-bit range.

Solution (Java)

```
class Solution {
    public int myAtoi(String s) {
        if (s == null || s.length() == 0) return 0;
    }
}
```

```

int i = 0;
int n = s.length();

// 1. Read in and ignore leading whitespace
while (i < n && s.charAt(i) == ' ') {
    i++;
}

if (i == n) return 0;

// 2. Check for sign
int sign = 1;
if (s.charAt(i) == '+' || s.charAt(i) == '-') {
    sign = (s.charAt(i) == '-') ? -1 : 1;
    i++;
}

// 3. Convert digits and handle overflow
long result = 0; // Use long to detect overflow
while (i < n && Character.isDigit(s.charAt(i))) {
    int digit = s.charAt(i) - '0';
    result = result * 10 + digit;

    // 4. Clamp to 32-bit signed integer range
    if (sign == 1 && result > Integer.MAX_VALUE) {
        return Integer.MAX_VALUE;
    }
    if (sign == -1 && -result < Integer.MIN_VALUE) {
        return Integer.MIN_VALUE;
    }

    i++;
}

return (int) (result * sign);
}

```

Why this works (the invariants)

- **Whitespace rule:** the first loop only skips ' ', matching the spec exactly.
- **Sign rule:** at most one character is consumed as a sign, and only if it appears before any digit.
- **Clamping rule:** every digit addition is bounds-checked against the signed 32-bit range, so the returned value is always clamped correctly — including the exact-boundary case `"-2147483648"`, where `-result == Integer.MIN_VALUE` (not `<`), so the loop keeps going and returns `MIN_VALUE` exactly.

Alternative: overflow check without `long`

If `long` is disallowed, check **before** multiplying:

```

if (result > Integer.MAX_VALUE / 10 ||
    (result == Integer.MAX_VALUE / 10 && digit > 7)) {
    return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
}

```

```
}
result = result * 10 + digit;
```

The `digit > 7` bound works because `MAX_VALUE = 2147483647` ; for the negative case, `-(result*10+digit)` clamps to `MIN_VALUE` when the magnitude exceeds 2147483647, and the same check is reused (digit 8 gives exactly `MIN_VALUE` , which is in range — clamping to `MIN_VALUE` there is acceptable for `"-2147483648"` ; more precisely you'd allow digit 8 for negative). The `long` version in the main solution is cleaner and equally accepted.

Complexity

- **Time:** $O(n)$ — one pass over the string.
- **Space:** $O(1)$ — only the accumulator and pointers.

Edge-case table (verify mentally)

Input	Output	Why
<code>"42"</code>	<code>`42`</code>	plain digits
<code>" -42"</code>	<code>`-42`</code>	leading spaces + sign
<code>"4193 with words"</code>	<code>`4193`</code>	stops at non-digit
<code>"words and 987"</code>	<code>`0`</code>	no leading digits
<code>"-91283472332"</code>	<code>`-2147483648`</code>	clamps to <code>MIN_VALUE</code>
<code>"2147483648"</code>	<code>`2147483647`</code>	clamps to <code>MAX_VALUE</code>
<code>"-2147483648"</code>	<code>`-2147483648`</code>	exact boundary, not clamped
<code>"+-12" / "+-1"</code>	<code>`0`</code>	only one sign allowed
<code>" + 413"</code>	<code>`0`</code>	sign must be immediately before digits
<code>"+", "- ", " "</code>	<code>`0`</code>	no digits
<code>"0032"</code>	<code>`32`</code>	leading zeros fine
<code>"00000-42a1234"</code>	<code>`0`</code>	digit loop stops at <code>`-`</code> after zeros
<code>"20000000000000000000"</code>	<code>`2147483647`</code>	clamps mid-loop, no <code>`long`</code> overflow

Interview traps

1. **Whitespace scope** — the spec says space only. `Character.isWhitespace` also skips `\t / \n` ; LeetCode's tests follow the spec, so both pass, but the stricter reading matches the problem statement.
2. **Clamping vs. wrapping** — Java `int` arithmetic wraps silently; never accumulate in `int` and clamp after. Detect during the loop.
3. **The negative boundary asymmetry** — `MIN_VALUE` 's magnitude (2147483648) is one more than `MAX_VALUE` (2147483647). The `-result < Integer.MIN_VALUE` comparison handles this exactly.
4. **What `result * sign` can overflow** — `result` is a `long` , so `result * sign` is `long` arithmetic; the cast to `int` happens after the value is already clamped. Safe.
5. **Follow-up the interviewer may ask:** "Do it without `long` ?" → the pre-check version above. "Stream it from a file?" → same state machine, process char by char.

Common mistakes

- Forgetting to check `i == n` after skipping whitespace → `StringIndexOutOfBoundsException`.
- Consuming the sign but forgetting to advance `i` past it (double-processes it as a non-digit — actually safe here, but worth noticing).
- Clamping only at the end using `int` arithmetic → silent wraparound; `"2147483648"` becomes `-2147483648`.
- Returning `0` for `"-2147483648"` by clamping with the wrong comparison (`-result <= MIN_VALUE` instead of `<`).